

Modular Programming Project in C

Application of Iterative and Decision-Making Constructs to Shapes and Physics

Name	Gurveer Singh
Batch	15
SAF ID	590032925

Aim

To develop a menu-driven modular C project for geometrical measurements and physical properties of sphere, cylinder, cone and cuboid using fundamental data types, call-by-value functions, selection statements and loops.

Project Files

File	Responsibility
main.c	Main menu, validation, solid selection and output
geometry.c	Surface area, volume and base-area functions
physics.c	Mass, weight, potential energy, pressure, buoyant force and buoyancy
math_utils.c	Newton-Raphson square-root calculation
solid.h	Constants and external function declarations

General Algorithm

1. Start and display the main menu inside a do-while loop.
2. Read and validate the menu choice. For geometry operations, ask for a solid and its dimensions.
3. Use switch-case to call the required function; all dimensions and densities must be positive and elevation must be non-negative.
4. Calculate geometry or physics result in the appropriate module. Do not repeat formulas in main.c.
5. Display the result using SI units and return to the menu.
6. Continue until option 10 (Exit) is selected, then stop.

Test Case Table

No.	Operation / Input	Type	Expected
1	Sphere area, r=2	Valid	50.2656 m ²
2	Cylinder volume, r=2, h=5	Valid	62.8320 m ³
3	Cone volume, r=3, h=4	Valid	37.6992 m ³
4	Cuboid base, l=4, b=3	Valid	12.0000 m ²
5	Radius = 0, then 2	Invalid/Boundary	Reject 0 and request again
6	Elevation = -1, then 0	Invalid/Boundary	Reject -1; accept 0
7	Base area + Sphere	Invalid choice	Message: base area not defined
8	Solid density 1000, fluid density 1000	Boundary	SUSPENDED
9	Solid density 800, fluid density 1000	Valid	FLOATS
10	Solid density 1200, fluid density 1000	Valid	SINKS

solid.h

```
#ifndef SOLID_H
#define SOLID_H

#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

double calculate_mass(double density);
double calculate_weight();
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);

double calculate_square_root(double number);

#endif
```

geometry.c

```
#include "solid.h"

double sphere_area(double r) {
    return 4 * PI * r * r;
}
double sphere_volume(double r) {
    return (4.0 / 3.0) * PI * r * r * r;
}
double cylinder_area(double r, double h) {
    return 2 * PI * r * (r + h);
}
double cylinder_volume(double r, double h) {
    return PI * r * r * h;
}
double cone_area(double r, double h) {
    double slant = calculate_square_root(r*r + h*h);
    return PI * r * (r + slant);
}
double cone_volume(double r, double h) {
    return (1.0 / 3.0) * PI * r * r * h;
}
double cuboid_area(double l, double b, double h) {
    return 2 * (l*b + b*h + l*h);
}
double cuboid_volume(double l, double b, double h) {
    return l * b * h;
}
double circular_base_area(double r) {
    return PI * r * r;
}
double cuboid_base_area(double l, double b) {
    return l * b;
}
```

math_utils.c

```
#include "solid.h"

double calculate_square_root(double number) {
    double old_value, new_value, difference;

    if (number < 0) return -1;
    if (number == 0) return 0;

    old_value = number;

    do {
```

```

        new_value = 0.5 * (old_value + number / old_value);
        difference = new_value - old_value;
        if (difference < 0)
            difference = -difference;
        old_value = new_value;
    } while (difference >= TOLERANCE);

    return new_value;
}

```

physics.c

```

#include <stdio.h>
#include "solid.h"

static double get_volume(void) {
    int solid;
    double r, h, l, b;

    do {
        printf("Solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): ");
        scanf("%d", &solid);
    } while (solid < 1 || solid > 4);

    if (solid == SPHERE) {
        do { printf("Radius: "); scanf("%lf",&r); } while (r <= 0);
        return sphere_volume(r);
    }
    if (solid == CYLINDER || solid == CONE) {
        do { printf("Radius: "); scanf("%lf",&r); } while (r <= 0);
        do { printf("Height: "); scanf("%lf",&h); } while (h <= 0);
        if (solid == CYLINDER) return cylinder_volume(r,h);
        return cone_volume(r,h);
    }

    do { printf("Length: "); scanf("%lf",&l); } while (l <= 0);
    do { printf("Breadth: "); scanf("%lf",&b); } while (b <= 0);
    do { printf("Height: "); scanf("%lf",&h); } while (h <= 0);
    return cuboid_volume(l,b,h);
}

double calculate_mass(double density) {
    return density * get_volume();
}

double calculate_weight() {
    double density;
    do { printf("Density (kg/m^3): "); scanf("%lf",&density); }
    while (density <= 0);
    return calculate_mass(density) * GRAVITY;
}

double potential_energy(double elevation) {
    double density;
    do { printf("Density (kg/m^3): "); scanf("%lf",&density); }
    while (density <= 0);
    return calculate_mass(density) * GRAVITY * elevation;
}

double calculate_pressure(double density) {
    int solid;
    double r,h,l,b,volume,area;

    do {
        printf("Solid (2 Cylinder, 3 Cone, 4 Cuboid): ");
        scanf("%d",&solid);
    } while (solid < 2 || solid > 4);

    if (solid == CYLINDER || solid == CONE) {
        do { printf("Radius: "); scanf("%lf",&r); } while (r <= 0);
        do { printf("Height: "); scanf("%lf",&h); } while (h <= 0);
        area = circular_base_area(r);
        if (solid == CYLINDER) volume = cylinder_volume(r,h);
        else volume = cone_volume(r,h);
    } else {
        do { printf("Length: "); scanf("%lf",&l); } while (l <= 0);
        do { printf("Breadth: "); scanf("%lf",&b); } while (b <= 0);
        do { printf("Height: "); scanf("%lf",&h); } while (h <= 0);
        area = cuboid_base_area(l,b);
        volume = cuboid_volume(l,b,h);
    }
    return density * volume * GRAVITY / area;
}

double buoyant_force(double fluid_density) {
    return fluid_density * get_volume() * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density) {
    if (solid_density > fluid_density) return SINKS;
}

```

```

    if (solid_density < fluid_density) return FLOATS;
    return SUSPENDED;
}

```

main.c

```

#include <stdio.h>
#include "solid.h"

int main() {
    int choice, solid, result;
    double r,h,l,b,density,elevation,fluid_density,solid_density,value;

    do {
        printf("\n----- MAIN MENU ----- \n");
        printf("1. Surface Area\n2. Volume\n3. Base Area\n4. Mass\n");
        printf("5. Weight\n6. Potential Energy\n7. Pressure\n");
        printf("8. Buoyant Force\n9. Buoyancy Behavior\n10. Exit\n");
        printf("Enter choice: ");
        scanf("%d",&choice);

        if (choice >= 1 && choice <= 3) {
            do {
                printf("Solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): ");
                scanf("%d",&solid);
            } while (solid < 1 || solid > 4);
        }

        switch (choice) {
            case 1:
                if (solid == SPHERE) {
                    do { printf("Radius: "); scanf("%lf",&r); } while(r<=0);
                    value=sphere_area(r);
                } else if (solid == CYLINDER || solid == CONE) {
                    do { printf("Radius: "); scanf("%lf",&r); } while(r<=0);
                    do { printf("Height: "); scanf("%lf",&h); } while(h<=0);
                    value=(solid==CYLINDER)?cylinder_area(r,h):cone_area(r,h);
                } else {
                    do { printf("Length: "); scanf("%lf",&l); } while(l<=0);
                    do { printf("Breadth: "); scanf("%lf",&b); } while(b<=0);
                    do { printf("Height: "); scanf("%lf",&h); } while(h<=0);
                    value=cuboid_area(l,b,h);
                }
                printf("Surface Area = %.4f m^2\n",value);
                break;

            case 2:
                if (solid == SPHERE) {
                    do { printf("Radius: "); scanf("%lf",&r); } while(r<=0);
                    value=sphere_volume(r);
                } else if (solid == CYLINDER || solid == CONE) {
                    do { printf("Radius: "); scanf("%lf",&r); } while(r<=0);
                    do { printf("Height: "); scanf("%lf",&h); } while(h<=0);
                    value=(solid==CYLINDER)?cylinder_volume(r,h):cone_volume(r,h);
                } else {
                    do { printf("Length: "); scanf("%lf",&l); } while(l<=0);
                    do { printf("Breadth: "); scanf("%lf",&b); } while(b<=0);
                    do { printf("Height: "); scanf("%lf",&h); } while(h<=0);
                    value=cuboid_volume(l,b,h);
                }
                printf("Volume = %.4f m^3\n",value);
                break;

            case 3:
                if (solid == SPHERE) {
                    printf("Base area is not defined for a sphere.\n");
                } else if (solid == CYLINDER || solid == CONE) {
                    do { printf("Radius: "); scanf("%lf",&r); } while(r<=0);
                    printf("Base Area = %.4f m^2\n",circular_base_area(r));
                } else {
                    do { printf("Length: "); scanf("%lf",&l); } while(l<=0);
                    do { printf("Breadth: "); scanf("%lf",&b); } while(b<=0);
                    printf("Base Area = %.4f m^2\n",cuboid_base_area(l,b));
                }
                break;

            case 4:
                do { printf("Density (kg/m^3): "); scanf("%lf",&density); }
                while(density<=0);
                printf("Mass = %.4f kg\n",calculate_mass(density));
                break;

            case 5:
                printf("Weight = %.4f N\n",calculate_weight());
                break;

            case 6:
                do { printf("Elevation (m): "); scanf("%lf",&elevation); }
                while(elevation<0);
                printf("Potential Energy = %.4f J\n",potential_energy(elevation));

```

```

        break;

    case 7:
        do { printf("Density (kg/m^3): "); scanf("%lf",&density); }
        while(density<=0);
        printf("Pressure = %.4f Pa\n",calculate_pressure(density));
        break;

    case 8:
        do { printf("Fluid density (kg/m^3): "); scanf("%lf",&fluid_density); }
        while(fluid_density<=0);
        printf("Buoyant Force = %.4f N\n",buoyant_force(fluid_density));
        break;

    case 9:
        do { printf("Solid density: "); scanf("%lf",&solid_density); }
        while(solid_density<=0);
        do { printf("Fluid density: "); scanf("%lf",&fluid_density); }
        while(fluid_density<=0);
        result=determine_buoyancy(solid_density,fluid_density);
        if(result==FLOATS) printf("The solid FLOATS.\n");
        else if(result==SINKS) printf("The solid SINKS.\n");
        else printf("The solid remains SUSPENDED.\n");
        break;

    case 10:
        printf("Program ended.\n");
        break;
    default:
        printf("Invalid menu choice. Try again.\n");
    }
} while(choice != 10);

return 0;
}

```

Compilation and Execution

```
gcc main.c geometry.c physics.c math_utils.c -o solid_project
./solid_project
```

Sample Output

```
----- MAIN MENU -----
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Potential Energy
7. Pressure
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
Enter choice: 1
Solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 1
Radius: 2
Surface Area = 50.2656 m^2

Enter choice: 9
Solid density: 800
Fluid density: 1000
The solid FLOATS.

Enter choice: 10
Program ended.
```

Result

The modular C project was prepared using separate source files, call-by-value functions, loops and selection statements. It calculates the required geometrical and physical quantities without arrays, pointers, recursion, structures or math.h, and uses Newton-Raphson iteration for square root.